15

Representations of user research

User research aims to inform design decisions. It aims to gather knowledge about the prospective users of a system, their activities, the contexts in which they work, and the existing technologies they use. Previous chapters in this part of the book have covered the *methods* used in user research. By contrast, this chapter is about the *outcomes* of user research, in particular how we *represent* research data using diagrams, models, and text. Figure 15.1 shows a couple of examples of such representations.

The previous chapters detailed how to analyze interviews, how to do ethnography, and how to analyze surveys using statistics. Why fuss further about representations of data? The main reason is that how we represent user research matters, just as how we represent a logical puzzle or lay out a math problem impacts how easy it is to solve. Data alone do not *do* anything; a dataset is simply a collection of observations. This chapter focuses on how we can represent data to gain information, uncover insights, and help us act. In particular, we care about the following applications of user research data:

- Summarization. Representations can consolidate extensive data from user research. For instance, Tychsen and Canossa [836] used metrics collected from a computer game to create representations of how players interact with the game. Those representations pulled together many metrics on how people engage with the game and clearly captured the higher-level goals of players (e.g., play styles).
- Spelling out requirements. The identified requirements may then be used in design briefs and software production contracts; see Chapter 36.
- Inspiration. This includes finding new opportunities and concepts to drive design.
- Communicating within a development team. For example, the results of usability tests are often summarized as criticality-ranked issues, which can be shown to developers together with example data such as videos (Chapter 43).
- Checking insights with stakeholders. For example, task analyses are essentially hypotheses about how users structure their activities and should be verified with stakeholders [24].

We may use user data for all of this; the techniques described in this chapter help us do it.

User research informs design, which is the topic of Part VI. That part of the book concerns the use of insights from user research to create ideas about future interactive systems. Some of the techniques that we discuss here can also be used for creating ideas for future interactive systems. For instance, we may represent the sequence of steps to be taken in a new system in a sequence model similar to that in Figure 15.1. However, these aspects of human–computer interaction (HCI) have different aims: User research describes current affairs, while design imagines possible futures. It is important to know whether a scenario is based on interviews (user research) or reflects an imagination of how things might work (design). One of the most critical challenges in design is to go beyond the familiar and propose a change, a new way of acting in the world. This requires

Introduction to Human-Computer Interaction. Kasper Hornbæk, Per Ola Kristensson, and Antti Oulasvirta, Oxford University Press. © Kasper Hornbæk, Per Ola Kristensson, and Antti Oulasvirta (2025).
DOI: 10.1093/oso/9780192864543.003.0015

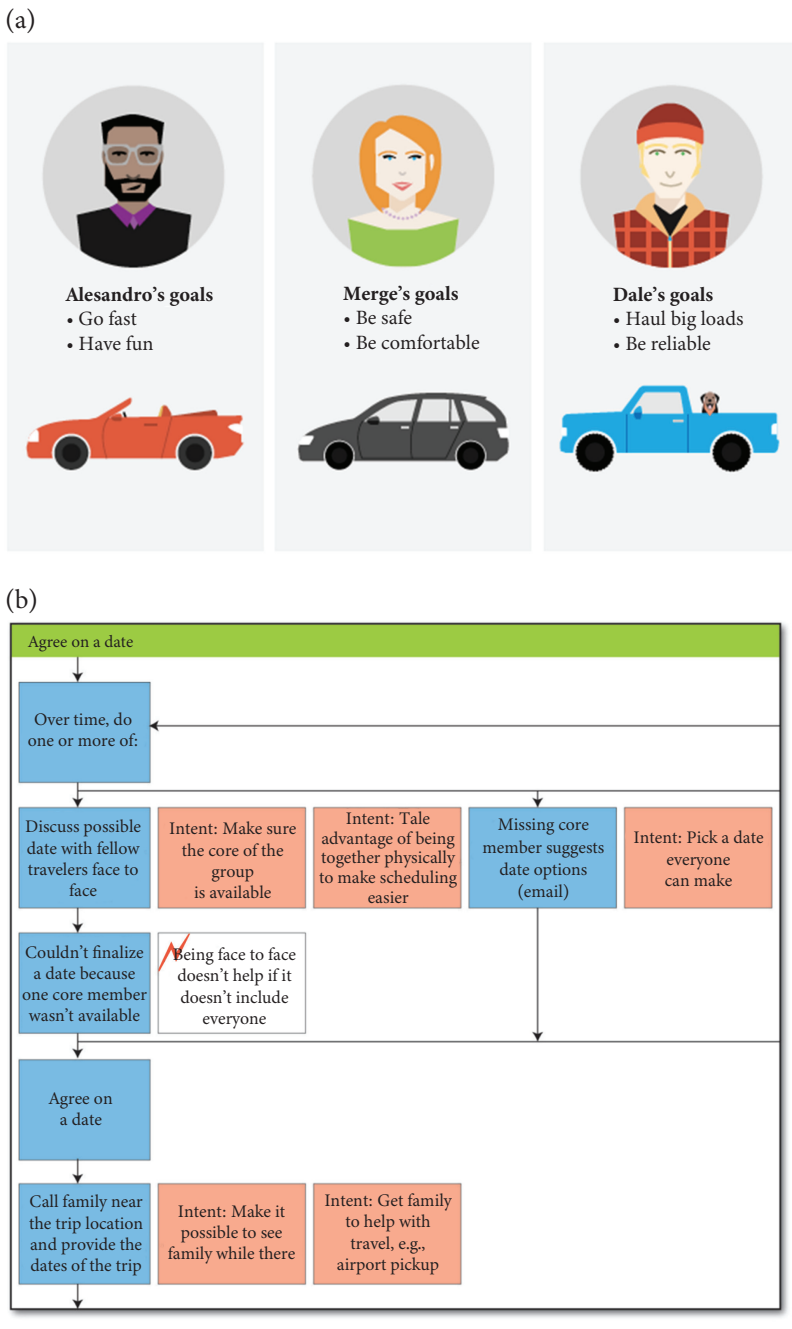


Figure 15.1 For user research to inform design and decision-making, the acquired data must be summarized in some useful way. The top panel shows a few simple personas, that is, representations of user groups as archetypes [169]; the bottom panel shows a summary of how people select a date to go traveling [348]. Both representation types help understand, apply, and communicate the findings of user research.

an understanding of the familiar, the starting point, and the constraints and expectations this challenge poses to the future.

User research also informs engineering, which we discuss in Part VII. User research obtains information on what users want, how they work, and what they value. In engineering processes, these are translated into requirements. Requirements, in turn, are used as objectives and criteria for deciding when a system is good enough. The results of user research can also be translated into algorithms, for example, as objectives and constraints in optimization and learning algorithms that drive intelligent user interfaces.

The results of user research are also part of collaborative efforts. As such, they have an important communicative purpose: They help form a shared understanding among stakeholders. To this end, user research has the obligation to be well-formed and well-communicated.

In this chapter, we discuss focal points for representing user data. We cover the following aspects: representing knowledge about users, including their needs, preferences, and capabilities; what users do and how these activities are organized; artifacts, including the influence of the physical layout of the work environment; and contexts, including understanding users' situations and use-defining factors. We also discuss user requirements for technical functionality and the performance of systems.

15.1 Representations of people

A big part of user research is understanding who the users are, what their needs are, what they try to achieve, and how technology might fit into their lives and work. At the beginning of this part (Section 10.2), we talked about establishing *who* the users are. For instance, we may define user segments based on market research, or we may spell out our initial ideas on who the users are and what their goals are. In this regard, representations of people can assist us in drawing actionable conclusions about users. Design and development teams need to be able to relate to users and make inferences about users on issues that might not have been covered by user research. The most popular tool for doing this is personas.

15.1.1 Personas

A *persona* is a description of an idealized (nonexisting) person who represents a group or type of users. The term was popularized by Cooper et al. [169] and has since been widely used in HCI. The idea is to construct archetypes of users in the form of fictional (but representative) individuals with specific characteristics. Personas are based on user research; as such, they are more a synthesis or aggregate than a fiction. Below, we report an example of two adult personas taken from Nielsen [594].

> Camilla and Jesper live on the outskirts of Copenhagen. They are 35 and 39 respectively, and they have enough on their plate with children and careers. They have lived together for the past five years. Two years ago, they had their son Storm. Jesper has two children from his previous marriage, Christian and Caroline, 11 and 8 years old. The children live with Jesper and Camilla every second week. Camilla and Jesper prefer to use self-service solutions, and they are curious about what information the public sector stores on them, and how it is stored.

Such descriptions of personas may be combined with pictures or sketches to make the personas more memorable, engaging, and concrete. Sociodemographic information can be added. Cooper

suggested that the goals of the personas should also be described. Nielsen [594] further suggested that personas be combined with scenarios. Scenarios may also be used without personas, as discussed in Section 15.2.1.

Personas can summarize general tendencies in the data, acting like a mode or median member in that group. They can also be stereotypifications of that group—embellished versions that help the researcher form perspectives and ensure sufficient diversity in design.

Representing user research as personas has several purposes. The first purpose is to create specific individuals. Personas can help avoid making decisions based on idealized users who are too flexible (or "elastic"). A degree of inflexibility helps to keep design in check. It ensures that when we make design decisions, we do not distort user data to benefit the design. Second, personas help avoid self-centeredness. Technology is too often developed for developers by developers. Third, personas prioritize data. Personas require selecting which user segments are most important in design. Fourth, personas synthesize data. Raw data are often too complex to deal with in design. A few well-selected personas can efficiently distill and communicate the essence of a dataset. Fifth, personas can drive empathy. It is easier to relate to another person's (even if hypothetical) viewpoint than to quantitative summaries.

Creating personas

There are numerous methods for creating personas. Pruitt and Grudin [678] created a rich persona template for software engineering projects. The template contains slots for describing, among others, the user's basic properties—their name, age, and occupation, a day in their life, work activities, goals, fears, the market size they represent, technology attitudes, and quotes. However, long templates entail the risk of creating catalogs of information with no clear relevance to design.

We believe that personas that go beyond observable attributes ("Jane is 25, she attends a university") to capture the drivers of such behaviors ("Jane is fascinated by theoretical physics") are valuable [169]. How to obtain such characteristics without conjuring them up? One method is as follows, adapted from Cooper et al. [169]:

1. Group participants in the data by their role. Parent, student, manager, and administrator are examples of roles.
2. Identify potential drivers of behavior in the dataset. Go beyond observable behaviors and try to identify *behavioral variables* such as those describing attitudes, skills, beliefs, motivations, and aptitudes. These variables should capture aspects of why people did what they did. They may be continuous (e.g., "motivated by social connections [weak–strong]" or "excited about a new product feature [weak–strong]") or binary (e.g., "uses Windows 11 [yes/no]").
3. Map observational data about participants to behavioral variables. Take snippets of the data (e.g., interview quotes) and associate them with behavioral variables, for example, using Post-it notes.
4. Identify significant clusters forming around the behavioral variables.
5. Synthesize the characteristics of roles. Find clusters where a certain role has a particular behavioral variable strongly represented; these are candidates for the key characteristics of that role.
6. Check for completeness. Ensure that a reasonable proportion of the data belonging to a user role is covered in the clusters that reflect its characteristics.
7. Select a persona type and name it.
8. Verbally explain the key attributes and behaviors rooted in the clusters.

The method requires iteration between observational data, roles, behavioral variables, and clusters. This method ensures the personas can be rooted back to observational data; the clustering of data ensures that one can trace back what a characteristic is based on.

Drawbacks of personas

The persona method is often haphazardly applied. From a statistical perspective, a persona is a summary statistic analogous to a mean, median, or mode in some distribution. However, it is not always applied from this perspective; the questions of (1) how to cluster users and (2) how to select such persons to represent whole groups are often dismissed. The consequence is that personas can be almost anything, and the connection to the original data can be broken.

Pruitt and Grudin [678] reported issues they encountered in the use of personas at Microsoft. Their persona descriptions were not believable; the personas were "designed by committee" or lacked a link to the original data. The personas were also poorly communicated, such as via full-blown poster-sized lists, and little effort was put into understanding their main points. There was also no understanding of how to use the personas since they were not properly linked to the different stages of product development.

Personas have a relationship with user groups. *User groups* are segments of the user population. Personas are often thought to represent user groups or segments. However, defining a user group is tricky. From a statistical perspective, it is a clustering problem. Given a population of m samples (users), each depicted by a feature vector x, the goal is to assign every sample (user) to one of n groups such that the distance of two samples within a group is minimized and the distance to other groups is maximized. Unfortunately, although this is generally understood, personas and groups are often created based on intuition rather than the systematic comparison of users or statistical analysis. This has the unwanted consequence that such representations can be unconvincing to stakeholders and—worse—mislead the design process. Systematic methods like the one presented above increase the likelihood of avoiding these issues. New data-driven approaches are also emerging, as illustrated in Paper Example 15.1.1.

Paper Example 15.1.1: Automated generation of personas from log data

Generating personas has traditionally been a labor-intensive process. To form a persona, data on users must be collected, for example, via surveys or focus groups. The data then need to be labeled, clustered, and so on. Despite the high amount of effort, there is no guarantee that the personas will represent well the targeted user population. Moreover, personas can become outdated.

Automated persona generation refers to the use of online datasets to computationally generate up-to-date personas. These descriptions are automatically kept updated as changes occur in the user population.

Jung et al. [399] presented a case for automatically generating personas from data available from social media. They analyzed data from users who engage online with Al Jazeera, a popular news channel. The data came from 180,000 users in 181 countries and encompassed over 30 million interactions with Al Jazeera's YouTube channel. Such large datasets would normally be impractical for persona development.

The authors presented a statistical method for identifying user segments from such datasets. First user interaction patterns are identified from the data. These patterns indicate which

continued

Paper Example 15.1.1: Automated generation of personas from log data *(continued)*

products the users use and how. These clusters are then linked to demographic data, such as age distributions. Finally, persona descriptions are created around these attributes and then enriched with descriptive features, such as names and photos, obtained from generic databases (e.g., typical names of people of a certain age). An example of a resulting persona in Al Jazeera's case is as follows: "Samantha is a 25-year old female living in the US. She likes to read articles about society, environment, and racism on a computer terminal. She usually watches about 2 minutes of video." Any number of such personas can be created while ensuring they are rooted in the original data. They can be augmented by representative comments posted online by this group. The personas can also be updated on demand.

15.2 Representations of activities

In previous chapters in this part of the book, we have covered how difficult it is to obtain valid and reliable insights about the activities in which people participate. This section describes how we can represent these insights in a way that helps to clarify people's tasks.

Before we start, we need to be clear about the terminology for activities. An *activity* is the total of what a user or a group of users wishes to achieve. It is purposeful interactions among actors that develop over time. A *task* is some piece of work to be done. As an analytic tool in thinking about user research, tasks have several applications:

1. The description of a task is often about things to be done, not about how they are done. When represented like this, tasks can be compared across different interactive systems. Part IV will explain why this is a slightly simplified picture by arguing that tasks and tools are always intertwined.
2. The description of a task can be a description of how things are done. HTA, as we learn here, involves the decomposition of a task (when carried out with a particular tool).
3. The description of a task can be specific and concrete.
4. A task is a meaningful unit. That is, getting a task done is a meaningful achievement for the user, who cares about being able to complete the task.

Next, let us look at some ways to work systematically with tasks.

15.2.1 Scenarios

Scenarios are narrative accounts of an activity or task. Scenarios use storytelling to communicate relationships between users and events. They are less structured than most analytical tools, such as task analysis or user requirements. Scenarios are almost always written from the user's point of view. This facilitates empathizing with users. Scenarios are almost never prescriptive in the sense of telling how things *should* occur. As a consequence of lacking a rigid predefined structure, scenarios can be underspecified as accounts of what happened. However, a well-written narrative may help us imagine the experience of a person from their perspective. Paper Example 15.2.1 provides an example of how explainable AI was studied using scenarios.

Example scenario by Rosson and Carroll [714]:

> After three years at Virginia Tech, Sharon has learned to take advantage of her free time in-between classes. In her hour between her morning classes, she stops by the computer lab to visit the science fiction club. She has been meaning to do this for a few days because she knows she'll miss the next meeting later this week. As she opens a Web browser, she realizes that this computer will not have her bookmarks stored, so she starts at the homepage of the Blacksburg Electronic Village. She sees local news and links to categories of community resources (businesses, town government, civic organizations). She selects "Organizations" and sees an alphabetical list of community groups. She is attracted by a new one, the Orchid Society, so she quickly examines their Web page before going back to select the Science Fiction Club page. When she gets to the club page, she sees that there are two new comments in the discussion on Asimov's Robots and Empire, one from Bill and one from Sara. She browses each comment in turn, then submits a reply to Bill's comment, arguing that he has the wrong date associated with discovery of the Zeroth Law.

Scenarios are written as stories. They contain the basic elements of a story: a setting, one or more actors with their goals and characteristics, and some tools and contexts that are available for that. The scenario then describes the sequence of actions and events that led to an outcome. It is important to describe the actors' thinking and experiences related to technology: how they set goals for, think about, react to, and experience it.

Paper Example 15.2.1: Scenario-based development of explainable AI

It has become almost a truism that AI systems must be designed to be trustworthy and transparent. Alas, realizing this vision has turned out to be difficult. One reason is that it is hard to explain to users the reasons behind AI's actions. Simple approaches to explaining AI, such as presenting a simplified model of the AI's reasoning to users, have failed: Users have very little interest in detailed accounts of how AI draws its conclusions. There is a need to rethink what explanations are and how they are interacted with.

Wolf [908] turned to scenario-based design to address this problem. They studied aging-in-place monitoring, that is, AI systems that help aging people maintain autonomy by monitoring their daily activities. Most of the time, these systems do not need to explain themselves; the question is when should they, and how?

The authors generated what they call explainability scenarios based on ethnographic studies of AI and machine learning professionals. In two field studies, they observed these professionals across three projects. To form the scenarios, the primary data were augmented with results reported in studies of caretakers and other stakeholders.

The following example scenario imagines the experiences of Jody and Catherine in using an aging-in-place system [p. 254]:

> Jody checks the app on her phone that displays her mom's daily activity report—the outcome of in-unit monitoring that captures and analyzes her mom's activities of daily living (ADLs). Her mom, Catherine, recently moved into an aging care facility and together with Jody decided to enroll in the facility's new activity monitoring program.
>
> [. . .]
>
> The system is designed to help keep Jody aware of her mom's daily activity levels (and alerting Jody of any abnormal changes or deviances in her mom's behavior) by providing daily reports in the app. To maintain Catherine's privacy, the reports do not document each

continued

Paper Example 15.2.1: Scenario-based development of explainable AI *(continued)*

and every activity of the day. Instead, it classifies her activity for the day into three levels (green, yellow, red) and provides a general explanation of why the system applied that label. [. . .]

Several days go by and Catherine's reports all turn up green. Then one day, Jody sees that her mom's been given a yellow report. "No kitchen activity recorded today." Jody calls her mom to ask how her day was and if anything was wrong. "I went for a long walk with my new neighbor, Vicki. It was nice, we have a lot in common. We ended up going out for lunch," Catherine explained. Jody was relieved to hear that "no kitchen activity recorded" wasn't anything alarming, she was actually excited to hear her mom was making friends.

The scenario continues to an event where these explanations do not suffice:

As Jody opens the app again she is greeted with a new message: "Your Monthly Activity Report is available." The report provides a dashboard view of Catherine's activity levels for the past month. It also provides a prediction—based on the predictive model from pilot data, the app has high confidence that seniors with activity levels similar to Catherine's will need to transition to an assisted living level of care within the next three months. Okay, Jody thinks, why would that be? She clicks on the "Tell me more" link next to the prediction, which says the top influences on the prediction are: age and frequency of ADLs. Her mom has mostly been getting "green" reports so Jody is confused why she might need escalated care. Unsure of who she is supposed to ask for help, she wonders if the app has more explanation on what to do with the Monthly Activity Reports. She looks around inside the app, and finds an FAQ list that mentions the Report. It describes the Reports contents and features a link to a demo video. She clicks on the video, but quickly recognizes it is an intro video she watched several weeks earlier when they were getting the system set up. At the bottom of the FAQ there is a "Feedback" form where she can send a message to the app developers. Unsure of what to do next, she calls Metria, the transition specialist at Catherine's facility. Metria explains that she would have to review the report with the data team to be able to answer Jody's questions. Metria says she will follow up as soon as she can.

The scenario can appear messy and excessively specific. However, it is exactly these qualities of inflexibility that make it relatable. For AI designers, the main takeaway is that one broad type of explanation may not suffice: There are several layers of explanations that need to be considered when needed. Moreover, explanations are discursive. Instead of a single decision-maker seeking explanations, Jody must talk with Catherine to establish what has happened. Therefore, technical measures such as "accuracy" may not be sufficient for developing explanations that actually work.

15.2.2 Customer journeys

Customer journey mapping is an account of events where a user encounters a service or product. The map is presented in the form of a journey—a path that goes through *touchpoints*. Scenarios often focus on events during the use of technology; customer journey mapping also covers events that precede the actual use, for example, advertisements, the creation of accounts, onboarding, and so on. The term "customer" depicts this view: Touchpoints are typically depicted from the perspective of the evolving relationship between a customer and a business, where a might-be customer transforms into a loyal customer via the touchpoints. The actual use of technology may only have a minor role in this. At every touchpoint, goals, thoughts, and experiences are reported.

Rosenbaum et al. [709] described a method for creating customer journey maps. It calls for the analysis of two dimensions:

1. Touchpoints, that is, the sequence of events through which customers interact with a service
2. Strategic actions associated with the touchpoints.

Touchpoints are typically divided into three periods: pre-service, service, and post-service. The pre-service period is about the user's experience before registering or purchasing the service. For example, one may encounter advertisements for the service or hear about it from friends. The post-service period takes place after the actual service, for example, when posting on social media about the experience or complaining about the service to customer service. Strategic actions are enabling actions that different participant groups should take to make the touchpoint successful. For example, strategic actions can concern what a manager, customer-facing personnel, or designer is required to do.

15.2.3 Task analysis

Task analysis is a method for decomposing tasks and presenting them as hierarchically organized sequences of subtasks. It decomposes and exposes the *structure* of interaction [24]. Task analysis is to HCI what requirements engineering is to software engineering. Many of the analytical evaluation methods we cover later in this book (Chapter 41) build on task analysis. Task analysis can also inform design; an example is given in Paper Example 15.2.2. One reason for its popularity is its systematicity. This also means that carrying out task analysis properly, especially when it includes validating the analysis with the stakeholders, is time-consuming.

Paper Example 15.2.2 Task analysis informs layout design

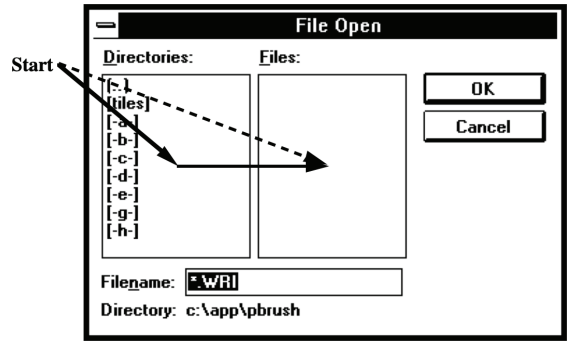
Task analysis is a systematic representation of users' activities. The method is very common in the study of safety-critical systems, where it is used to identify potential errors and needs for training personnel. In user-centered design, task analysis forms the basis of several analytical evaluation methods, such as cognitive walkthrough and keystroke-level model (see Chapter 41). Task analysis can also inform low-level decisions in design.

Sears [753] presented *layout appropriateness* (LA), a metric to inform the design of widget layouts. A widget layout is a graphically laid out set of interactive widgets, such as text

continued

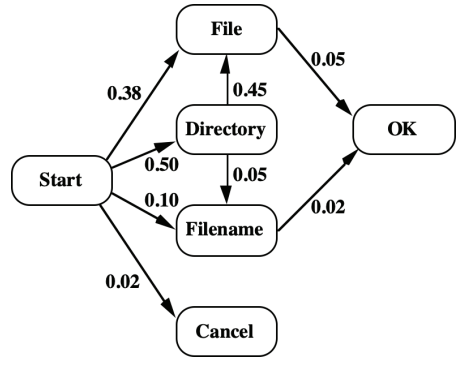
Paper Example 15.2.2 Task analysis informs layout design *(continued)*

fields, scrollbar, toggles, buttons, and so on. To compute LA, two inputs are needed: (1) task sequences that describe users' widget-level actions and (2) estimations of their frequencies. The former can be obtained via task analysis, the latter via questionnaires, logs, or by asking designers or stakeholders to rate the priority of tasks. This knowledge is expressed in a transition graph where nodes are widgets and links transition probabilities. For example, the following graph shows an analysis of the task of opening a file in a word editor.



Link labels denote how probable each transition is.

Layout appropriateness can be computed by weighing the cost of each sequence by its frequency. The cost of a sequence can be estimated from the distance of mouse movements or eye movements. Layouts that favor frequent user tasks receive a better (lower) LA score. The author presented a method to optimize a given widget layout to optimize its LA score. An LA-optimal layout represents the optimal cost-minimizing widget layout for the given set of user tasks. It can be obtained using algorithmic methods of optimization (Chapter 39). For example, the following figure shows the LA-optimal widget layout for the task of opening a file:



Designers can use the LA-optimal layout as a starting point for the design of a graphical interface. They can use it to assess how far their current design is from the optimal design (e.g., 95% LA-optimal).

Task analysis provides a *functional description* of behavior. It begins by outlining goals before considering actions to accomplish the task:

> Complex tasks are defined in terms of a hierarchy of goals and subgoals nested within higher order goals, each goal, and the means of achieving it being represented as an operation. The key features of an operation are the conditions under which the goal is activated and the conditions which satisfy the goal together with the various actions which may be deployed to attain the goal. These actions may themselves be defined in terms of subgoals. [24, p. 68]

The output of task analysis is a *task model*. It describes the requirements for carrying out a task successfully. It can be expressed in multiple ways: as a sequence or as a hierarchical diagram.

A *sequential task model* describes a task as a linear progression of subtasks. It is often described verbally. For example, a task model for logging into a web service could consist of the following steps:

1. Enter login name
2. Enter password
3. Click "Login."

How does one "enter" something? The first two rows can be expanded into two more subgoals, resulting in a higher-fidelity task model:

1. Click "Username" field with mouse
2. Type login name with keyboard
3. Click "Password" field with mouse
4. Type password with keyboard
5. Click "Login" with mouse.

One limitation of linear models is that the relationship between clicking and typing is not captured well. Hierarchical task analysis (HTA) can better account for this.

HTA is an established task analysis method that assumes two kinds of relationships between subtasks: (1) order, for example, task A precedes task B, and (2) part–whole relationship, for example, task B is a subtask of task A. Every subtask is thought to require operations to complete it. Operations are any actions that the user must take. An example of a simple task is given in Figure 15.2.

The core concept of task analysis is a *task*. The main task is recursively split into *subtasks*. A task (or a subtask) consists of a well-defined beginning state, goal state, and operations that transform the beginning state into the goal state. Some operations are conditional, that is, they can only be executed when some conditions apply. How many levels of subtasks one wants to use is a practical consideration.

Task analysis can be done empirically or analytically. The goal of *empirical task analysis*, which is used in user research, is to understand tasks in *unconstrained, real-world scenarios*. Typical data collection methods include interviews, the analysis of systems and their documentation (e.g., manuals), and observations in the field recorded via videos and field notes. In *analytical task analysis*, the modeling is speculative; it is carried out as part of design or evaluation. It can expose potential problems, such as complicated task structures or insufficient training. Task analysis is a modeling activity that relies on human analysts and cannot yet be adequately automated, though some attempts have been made, such as using data mining techniques on log data.

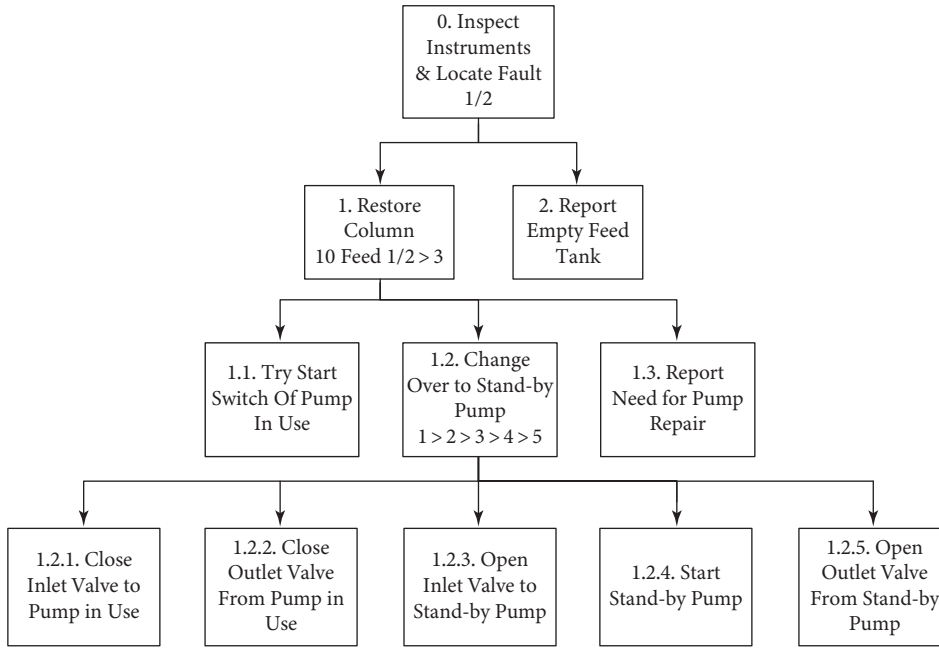


Figure 15.2 Hierarchical task analysis for the task of inspecting faults. Adapted from [24].

To conduct a HTA, one can start with a sequential model of the highest-level subtask. First, split the main task into a sequence of subtasks following each other. Consider a vending machine: You first need to decide which soda to buy, then find the corresponding button, then decide the payment method, and so on. Second, subtasks that are mutually dependent can be clustered to form a hierarchical structure.

The general steps in task analysis are [24]:

1. Decide the purpose(s) of the analysis
2. Achieve agreement between stakeholders on the definition of task goals and criterion measures
3. Identify the sources of task information and select the means of data acquisition
4. Acquire the data and draft a decomposition table or diagram
5. Check the validity of the decomposition with the stakeholders
6. Identify significant operations in light of the purpose of the analysis
7. Generate and, if possible, test hypotheses concerning the factors affecting learning and performance.

In addition to sequences and hierarchies, HTA offers notation to express *plans*, *conditions*, and *parallelisms*. Plans are routine procedures followed by people: "Do this, then this, then this . . ." Conditions are if–then rules that describe conditions for actions: "If condition X, do this." Parallelisms are subtasks that can be pursued in parallel. Moreover, stop rules can be expressed to tell a user when to stop executing a subtask. The conditions and the stop rules are annotated in the diagram.

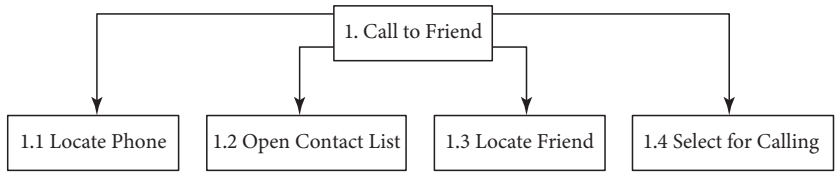


Figure 15.3 Hierarchical task analysis diagram for the task of calling a friend.

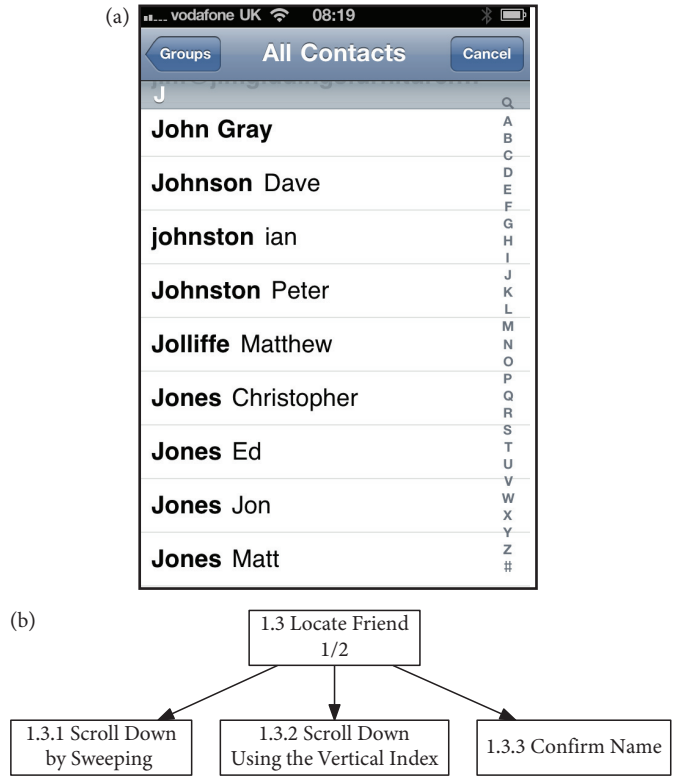


Figure 15.4 Diagram for the subtask of locating a friend in the contacts list. HTA introduces conventions to mark the order in which subtasks can be executed. Here, "1/2" means that either 1.3.1 or 1.3.2 is executed.

Example

Let us consider the task of calling a friend using a smartphone. Figure 15.3 shows a diagram with four subtasks ordered sequentially. To complete a subtask, more operations need to be carried out. For example, subtask "1.3 Locate friend" can be split as shown in Figure 15.4.

Like in Figure 15.4, one can continue splitting down the subtasks, noting their order and other important details. For example, subtask 1.3.2 involves the following steps:

1. Point at the vertical bar
2. Drag down finger
3. Confirm the right letter.

When we continue to do this for all subtasks all the way to the third level, we obtain a full HTA diagram.

15.3 Representations of context

A *contextual inquiry* (Chapter 11) offers a rich set of models to describe contexts and situations. Note that these are conceptual models, not models in the mathematical sense. They describe dependencies and causal relationships verbally or diagrammatically. A key concept is that of an *actor*. An actor can be anything—human or nonhuman, digital or physical—that participates in some active sense in the activity. A context model describes flows, sequences, artifacts, and the cultural and physical circumstances of the actors.

A *flow model* describes the main actors in the context and their relationships to the user. A flow model is drawn as a graph where actors are nodes and actions are directional arrows connecting them. The direction of an arrow indicates who takes the initiative. Arrows are annotated to describe what happens.

A *sequence model* is an ordered list of actions that need to be carried out to complete a task, that is, to get from an initial state to a goal state. The path to the goal state can have multiple subgoals. A sequence model provides a less rich representation of tasks than an HTA tree. Specifically, a sequence model provides an ordered list of subtasks equivalent to one level of depth in an HTA tree.

An *artifact model* describes interactions among artifacts in a workflow. An artifact can be a physical or digital object, such as a paper note, a social media message, a presentation template, or anything that is interacted with during work. In a contextual inquiry, such artifacts are documented, and the way they are manipulated and passed on is traced and modeled. Some artifact models are just collections of descriptions of artifacts. Sometimes, they are organized into descriptions of workflows.

A *cultural model* is an expression of the different beliefs, values, and practices of the actors. The model is typically drawn as a Venn diagram, where each circle corresponds to some designation of culture, such as data-focused, opportunity-driven, risk-averse, and so on.

A *physical model* represents the physical space and the actors' movements in the space.

15.3.1 Rich pictures

A rich picture is a way of representing the insights from user research in a diagram. It highlights the relationships between people and technologies and the richness of the use situation. Rich pictures were devised as part of the soft systems methodology and have been used in HCI by, among others, Monk and Howard [556].

Figure 15.5 shows an example of a rich picture. An effective rich picture shows some details of the structure of the situation, some of the key processes, and some of the concerns of the stakeholders. A rich picture can quote the actors or use any other means of communication. Rich pictures have no set syntax or set of notations. Rich pictures synthesize data from user research.

The process of creating a rich picture is as follows:

1. Gather people who are familiar with the research topic.
2. Outline the key stakeholders, including people and organizations, and some of the structure (e.g., geographical, physical, organizational).
3. Detail processes and concerns related to the identified stakeholders and structures.

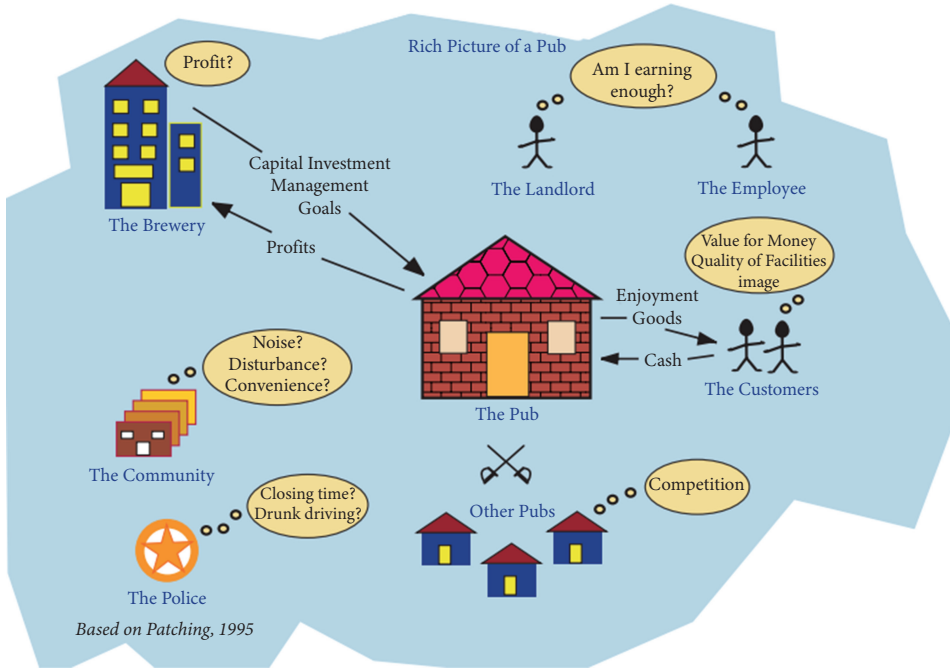


Figure 15.5 A rich picture of a pub [556]. The picture shows the main stakeholders, some of their considerations, and some links between them.

Remember that a rich picture is about capturing some of the complexity of a situation; it is not about describing a full system or the solution to a problem.

15.4 Requirements

User requirements are used in software engineering to specify users' expectations on what some software can do; see Chapter 36. Unlike system requirements, which express features with no necessary reference to users, user requirements are user-centered. They express criteria for the system's functioning from the user's perspective. For example, the requirements for a medical device might include being able to report when a sensor or the battery is not operational or allowing for quick and hygienic cleaning. Requirements can be expressed in many ways, such as non-technically for a client or technically for a developer. Normally, requirements are prioritized from both the business and end-user perspectives.

User requirements must be based on data and traceable. Triangulation is commonly recommended for identifying requirements. Different data sources provide complementary information that should be cross-checked when defining requirements. For example, a contextual inquiry may provide background information for a requirement, which is then elaborated via a focus group or interviews. In large software projects, the elicitation of requirements may involve interviews, observations, focus groups, workshops, brainstorming, and even prototyping. In some projects, requirements are gathered throughout, first during product launch and then via user feedback. In some cases, user requirements are reviewed by the users themselves. Confirming requirements

with stakeholders is generally recommended, especially in professional domains. Requirement traceability means that we know which observations each requirement is based on.

Each requirement should be associated with *verification criteria*, that is, a way of identifying whether a product fulfills the requirement. A verification criterion may be empirically defined, for example, "95% of representative users are able to complete task A within time T with no significant error." Vague criteria such as "Feature F is easy to use" are problematic because they leave their fulfillment open to disagreement. Being as precise as possible is important because user requirements can be contractual: A client and the developer agree on what a system must be able to do via requirements. After agreeing on the user requirements, the client is not expected to demand new features. On the other hand, the product is not considered ready if it does not meet the specified requirements. User requirements are also essential for planning, management, and communication in complex projects.

Use cases are a technique for eliciting and specifying requirements. The concept of use case emerged in the mid-1990s to help develop interactive systems. In software engineering, use cases are also used to denote cases where the user is not human, such as an external system. *User stories* are informally expressed use cases. They are narrative-like descriptions of system use written from the perspective of an end-user.

A use case outlines the actions that a user must take to achieve a goal. Several frameworks have been proposed for expressing them; at minimum, a case should state the initial state or preconditions for the case to occur, the user's goal, the flow of events, and the post-conditions—anything that must be done after achieving the goal. *Abstract use cases* define user intents but not sequences (flows). Use cases are expressed diagrammatically in a tabular, storyboard, or verbal format. Similar to user requirements, use cases are often associated with verification criteria.

15.5 Can representations be both valid and informative?

Representations of user research data are supposed to inform practical decisions. How do personas, scenarios, task analyses, and so on actually achieve this? They achieve this through two quite different routes, the realist route and the instrumentalist route, which are challenging to balance in user research.

According to the realist view, user research produces empirically grounded statements about users. A representation of user data is essentially a proposition, a claim about users. For instance, the claim that "User Segment A has a high need for security" can be true or false. Making valid statements about users is important for design and engineering efforts. Decisions about which features to include or how to design can be thought of as hypotheses about users: When the hypotheses are wrong, the design will fail.

According to the instrumentalist view, user research has instrumental value in inspiring design. The results of user research do not need to be valid in the realist sense—whatever helps design is valuable. In creative efforts where the goal is to envision novel possibilities, it may be detrimental to be tied too tightly to reality. From this viewpoint, even exaggerations and data embellishments are fine as long as they inspire new ideas. According to this view, user research does not need to seek a firm connection between a representation and the world it represents.

In practice, user research often serves both purposes. In projects where the goal is to inform decisions, observational data are routinely selected, redescribed, and augmented with other available knowledge. For example, we saw this in how personas and scenarios are created. Data are also interpreted. An analyst may flag something as surprising or write useful notes about users. Such

inferential leaps, even if small, pose a challenge. Leaps may be necessary to produce knowledge that informs practical decisions; however, such leaps may dissociate the analyst from observational data, making their conclusions less grounded. Practical decisions that are based on unsubstantiated or subjective inferences may be not only wrong but also damaging; for example, they may disregard or simplify certain user groups.

How to balance these two approaches? We hold that verifiability and traceability are important in all user research projects, even those with instrumentalist goals. *Verifiability* refers to how well a claim can be cross-checked with observations that are independent of the original dataset. For example, in task analysis, it is generally recommended to have task analysis diagrams checked by the stakeholders whose work it describes. If verification is hard or impossible, it is important to be able to trace the reasoning that led to the interpretation. *Traceability* refers to the documentation of the reasoning that led from the original data to a claim. For example, when writing up personas, one should document the data and other knowledge they were based on. Good practices like these lend credibility to user research and ensure that stakeholders pay heed to its results. Traceability is also important for accountability and the proper representation of the people involved.

Summary

- Representations capture findings and reflections in user research, aiding design.
- Representations summarize basic aspects of user data: users, events, tasks, activities, and contexts.
- Representations should be traceable and verifiable.

Exercises

1. Representation techniques. Consider a recent product launched by an IT company. Produce the following representations for the product: (a) a persona, (b) a scenario, (c) a task analysis diagram, (d) a customer journey map, and (e) a rich picture diagram.
2. The value of representations. If you have original data, why bother with second-order representations like personas and scenarios? Are we not always bound to lose information with representations? Discuss the pros and cons of using representations.
3. Large language models and user research. Large language models can be used to generate personas and scenarios. For example, you can prompt ChatGPT to generate a persona with the desired characteristics or even with real transcripts from interviews. Discuss how valid and informative such representations can be, including their pros and cons.